Services

1. Monolithic Architecture

What is Monolithic Architecture?

A **monolithic application** is a single application where **all functionalities are developed, deployed, and run together as one unit**.

Everything is inside one project.

For example,

An online banking application contains

- Login
- Customer Management
- Accounts
- Loans
- Cards
- Transactions
- Notifications
- Reports

All inside one Spring Boot project.

```
Bank Application

├── Login Module
├── Customer Module
├── Account Module
├── Loan Module
├── Card Module
├── Transaction Module
├── Notification Module
└── Database
```

Only **one application** is deployed.

Company Perspective

Imagine you're working at **ABC Bank**.

Initially, the bank has only **5 developers**.

Customers: **500**

Services:

- Login
- Account
- Loan

One Spring Boot application is enough.

```
ABC Bank

        Customers

            |

      Bank Application

-------------------------------

Login

Accounts

Loans

Database

-------------------------------
```

Everything works perfectly.

Spring Boot Monolithic Project Structure

```
bank-app

src

 ├── controller

 ├── service

 ├── repository

 ├── entity

 ├── dto

 ├── config

 └── BankApplication.java
```

Only one `pom.xml`

Only one `application.properties`

One database

One deployment

Advantages

✔️ Easy to develop

✔️ Easy to deploy

✔️ Easy debugging

✔️ One project

✔️ Less infrastructure

Good for

- College projects
- Small startups
- Internal tools

Problems Start as Company Grows

Now imagine after 5 years.

ABC Bank becomes very popular.

Customers increase

500

↓

5 Million

New services added

- Credit Cards
- UPI
- Insurance
- Mutual Funds
- Investments
- Chat Support
- AI Assistant

The application becomes huge.

```
Bank Application

--------------------------------------

Login

Accounts

Loans

Cards

Insurance

UPI

Reports
```

```
AI

Notifications

Investments

Transactions

---------------------------------------
```

Now there are

150 Developers.

Everyone works on the same project.

Problems of Monolithic Architecture

1. Huge Codebase

Project becomes

```
3 Million Lines of Code
```

Finding a bug becomes difficult.

---

2. Slow Development

Suppose you're working on the Loan module.

Your teammate is modifying the Account module.

Another developer changes Login.

Everyone pushes code to the same project.

Conflicts happen frequently.

---

3. Entire Application Must Be Redeployed

Suppose you fix only one bug.

```
Loan Interest Calculation
```

Even though only one class changed,

you must redeploy

```
Entire Bank Application
```

This increases downtime.

---

4. One Module Failure Can Affect Others

Suppose Notification Service crashes.

Sometimes the whole application may become unstable because everything is packaged together.

---

5. Scaling Problem

Suppose

Account module receives

```
10000 Requests/min
```

Loan module receives

```
100 Requests/min
```

But you cannot scale only Account.

You must create another copy of the **entire application**.

```
Copy 1

Login
```

```
Accounts

Loans

Cards

Insurance

--------

Copy 2

Login

Accounts

Loans

Cards

Insurance
```

Huge waste of resources.

Real Company Example

Imagine **Amazon**.

Millions of users.

Different teams handle

- Orders
- Payments
- Cart
- Reviews
- Delivery
- Search
- Inventory

Can all developers work inside one Spring Boot project?

Impossible.

That's why companies moved to Microservices.

2. Microservices Architecture

Definition

A **microservice** is a small, independent application that performs one specific business function and can be developed, deployed, and scaled independently.

Each service has its own

- Business logic
- Database (often)
- Deployment
- Team

Banking Example

Instead of one application,

split into multiple applications.

```
Login Service

Account Service

Loan Service

Card Service

Notification Service

Payment Service
```

Each is a separate Spring Boot project.

Architecture

```
                    Customers

                        |

-------------------------------------------------

 Login Service

 Account Service

 Loan Service

 Card Service

 Payment Service

 Notification Service

-------------------------------------------------
```

Each service runs independently.

Company Perspective

ABC Bank now has

500 Developers.

The company creates teams.

Team	Responsible For
Team A	Login
Team B	Accounts
Team C	Loans
Team D	Cards
Team E	Payments

Each team owns one microservice.

They can develop independently.

Independent Deployment

Suppose Loan Service has a bug.

Old way

Deploy entire bank application.

Microservices

Deploy only

```
Loan Service
```

Everything else keeps running.

Independent Scaling

Suppose

Account Service

```
20000 requests/min
```

Loan Service

```
100 requests/min
```

Scale only Account Service.

```
Account

Instance 1

Instance 2

Instance 3

Instance 4
```

```
Loan

Instance 1
```

Saves money.

Independent Database

Monolithic

```
One Database
```

Microservices

```
Account Database

Loan Database

Customer Database

Card Database
```

Each service controls its own data.

Advantages

✔️ Independent deployment

✔️ Independent scaling

✔️ Smaller codebase

✔️ Faster development

✔️ Better fault isolation

✔️ Technology flexibility (where appropriate)

Challenges

❌ More projects

❌ Inter-service communication

❌ Distributed transactions

❌ Monitoring

❌ Service discovery

❌ Security

Problem Without an API Gateway

Suppose we have:

```
Account Service

localhost:8081

Loan Service

localhost:8082

Card Service

localhost:8083
```

The frontend must know every URL.

```
React

↓

8081

↓

8082

↓
```

```
8083
```

If the ports change, the frontend breaks.

Also, authentication would need to be implemented in every service.

API Gateway

An API Gateway is the single entry point for all client requests.

Instead of clients talking directly to every microservice, they communicate only with the gateway.

```
Client

      |

API Gateway

  |

----------------------------

Account Service

Loan Service

Card Service

----------------------------
```

The gateway can perform:

- Routing
- Authentication
- Authorization
- Rate limiting
- Logging
- Load balancing (with service discovery)

- Request/response filtering

Example routes:

```
/accounts/**  -> Account Service
/loans/**     -> Loan Service
/cards/**     -> Card Service
```

Problem Without Eureka

Suppose the Account Service moves.

Earlier:

```
localhost:8081
```

Now:

```
localhost:9090
```

Every consumer must update its configuration.

This becomes impractical with many services.

Eureka Discovery Server

Eureka acts like a phone directory for microservices.

Instead of hardcoding addresses, services register themselves.

```
Account Service

↓

Registers

↓
```

```
Eureka Server

Loan Service

↓

Asks Eureka

↓

"Where is Account Service?"

↓

localhost:9090

↓

Call Successful
```

When an instance goes down or a new one starts, Eureka updates the registry automatically.

Overall Architecture

```
               Client

                  |

            API Gateway

                  |

           Eureka Server

       /        |        \

 Account Service Loan Service Card Service

      |            |           |

 Account DB      Loan DB      Card DB
```